# Autoencoders

## Nonlinear Dimensionality Reduction via Neural Reconstruction

Embeddings & Dimensionality Reduction

Constructor University

April 10, 2026

Lecture 3 of the Embeddings & Dimensionality Reduction series

Following: PCA $\rightarrow$ t-SNE $\rightarrow$ UMAP $\rightarrow$ **Autoencoders**

# Outline

# Where We Are in the Course

**Methods covered so far**

## Linear (PCA)

- Closed-form, eigendecomposition
- Parametric — generalises to new points
- **Cannot** capture curved manifolds

## Graph-based (t-SNE, UMAP)

- Captures nonlinear structure
- Preserves local neighbourhoods
- **Non-parametric** — no encoder for new points

## The gap: Autoencoders fill it

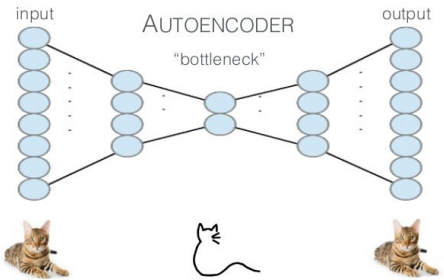
- ✓ **Nonlinear** (like t-SNE/UMAP)
- ✓ **Parametric** (like PCA)
- ✓ **Reconstruction** map back to input space
- ✓ **Generative** (VAE variant)
- ✓ **Scalable** to large datasets

## The autoencoder pipeline

$$\underbrace{\boldsymbol{x} \in \mathbb{R}^d}_{\text{high-dim input}} \xrightarrow{f_\phi} \underbrace{\boldsymbol{z} \in \mathbb{R}^k}_{\substack{\text{bottleneck} \\ k \ll d}} \xrightarrow{g_\theta} \underbrace{\hat{\boldsymbol{x}} \in \mathbb{R}^d}_{\text{reconstruction}}$$

- **Encoder** $f_\phi$: compresses $\boldsymbol{x}$ to latent space $\boldsymbol{z}$
- **Bottleneck**: forces the network to be selective
- **Decoder** $g_\theta$: reconstructs $\hat{\boldsymbol{x}}$ from $\boldsymbol{z}$
- **Training**: minimise $\|\boldsymbol{x} - \hat{\boldsymbol{x}}\|^2$

*Filtraction: anything not reconstructable from k (bottleneck) is discarded.*



## Definition

An **autoencoder** is a pair of parametric functions

$$f_\phi : \mathbb{R}^d \to \mathbb{R}^k, \qquad g_\theta : \mathbb{R}^k \to \mathbb{R}^d,$$

where $\phi$, $\theta$ are learnable parameters, $k < d$ is the *bottleneck dimension*.

**Encoder**:

$$f_\phi(\boldsymbol{x}) = \sigma_L(\boldsymbol{W}_L \cdots \sigma_1(\boldsymbol{W}_1 \boldsymbol{x} + \boldsymbol{b}_1) \cdots + \boldsymbol{b}_L)$$

**Common activations $\sigma$:**

- ReLU: $\max(0, x)$ — hidden layers
- Tanh — hidden layers
- Sigmoid — output (if $\hat{\boldsymbol{x}} \in [0, 1]$)
- Linear — output (real-valued)

# Reconstruction Loss

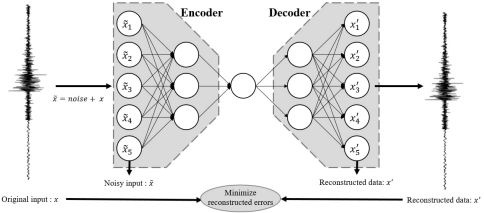
**Objective:** minimise average reconstruction error

$$\mathcal{L}(\phi, \theta) = \frac{1}{n} \sum_{i=1}^{n} \mathcal{L}_{\text{rec}}\big(\boldsymbol{x}_i,\ g_\theta(f_\phi(\boldsymbol{x}_i))\big)$$

## Mean Squared Error (MSE)

$$\mathcal{L}_{\text{rec}} = \|\boldsymbol{x} - \hat{\boldsymbol{x}}\|^2$$

Use for: real-valued, approximately Gaussian data



# Reconstruction Loss

**Objective:** minimise average reconstruction error

$$\mathcal{L}(\phi, \theta) = \frac{1}{n} \sum_{i=1}^{n} \mathcal{L}_{\text{rec}}\big(\boldsymbol{x}_i,\ g_\theta(f_\phi(\boldsymbol{x}_i))\big)$$

## Mean Squared Error (MSE)

$$\mathcal{L}_{\text{rec}} = \|\boldsymbol{x} - \hat{\boldsymbol{x}}\|^2$$

Use for: real-valued, approximately Gaussian data

## Binary Cross-Entropy (BCE)

$$\mathcal{L}_{\text{rec}} = -\sum_j \big[x_j \log \hat{x}_j + (1-x_j) \log(1-\hat{x}_j)\big]$$

Use for: binary / $[0, 1]$-bounded features

# Relationship to PCA

## Theorem

*A linear autoencoder (linear activations, MSE loss) trained to convergence learns a latent space that **spans the same subspace** as the top-k principal components.*

**Proof sketch:**
The loss $\|\boldsymbol{x} - \boldsymbol{W}_D \boldsymbol{W}_E \boldsymbol{x}\|^2$ is minimised when $\boldsymbol{W}_D \boldsymbol{W}_E = \Pi_k$ (the PCA projector).

Individual weights have rotational ambiguity, but the *subspace* is uniquely determined.

## Key insight

**PCA** = AE with
linear activations
+ MSE loss

Nonlinear activations
strictly generalise PCA

# Why the Bottleneck Enforces Structure?

**Without bottleneck** ($k \geq d$, linear):

- Model learns identity map $\hat{\boldsymbol{x}} = \boldsymbol{x}$
- Zero loss, but zero compression or insight

**With bottleneck** ($k \ll d$):

- Must discard information not reconstructable
- Gradient descent selects most informative directions
- Optimal $k$ = intrinsic dimensionality of data

Swiss Roll ($d = 3$, intrinsic dim $= 2$)

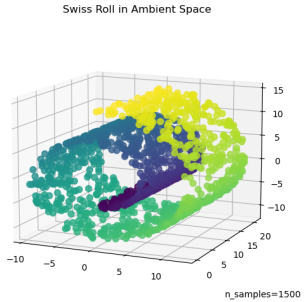
# Why the Bottleneck Enforces Structure

**Without bottleneck** ($k \geq d$, linear):

- Model learns identity map $\hat{\boldsymbol{x}} = \boldsymbol{x}$
- Zero loss, but zero compression or insight

**With bottleneck** ($k \ll d$):

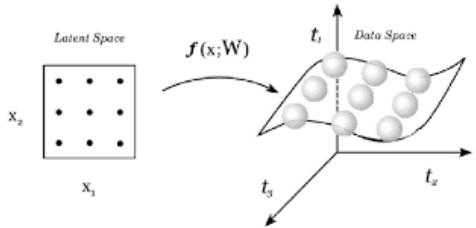
- Must discard information not reconstructable
- Gradient descent selects most informative directions
- Optimal $k$ = intrinsic dimensionality of data




Swiss Roll ($d = 3$, intrinsic dim $= 2$)
Reconstruction error decreases with increase of k

## Core assumption

Real high-dimensional data concentrates near a *low-dimensional manifold* $\mathcal{M} \subset \mathbb{R}^d$.
The AE learns a **structure** of this manifold.




*Simple geometric operations in $\mathcal{Z}$ (interpolation, clustering) correspond to semantic operations in $\mathcal{X}$.*

# Mini-Batch Training Algorithm

**Algorithm 1** AE mini-batch SGD

**Require:** $\{\boldsymbol{x}_i\}$, lr $\eta$, batch size $B$

**for** each epoch **do**
  **for** each mini-batch $\mathcal{B}$ **do**
    $\boldsymbol{z}_i \leftarrow f_\phi(\boldsymbol{x}_i)$ *// encode*
    $\hat{\boldsymbol{x}}_i \leftarrow g_\theta(\boldsymbol{z}_i)$ *// decode*
    $\hat{\mathcal{L}} \leftarrow \frac{1}{B}\sum \|\boldsymbol{x}_i - \hat{\boldsymbol{x}}_i\|^2$
    Backprop through $g_\theta$ then $f_\phi$
    Update $\phi$, $\theta$ with Adam
  **end for**
**end for**

**Practical tips:**

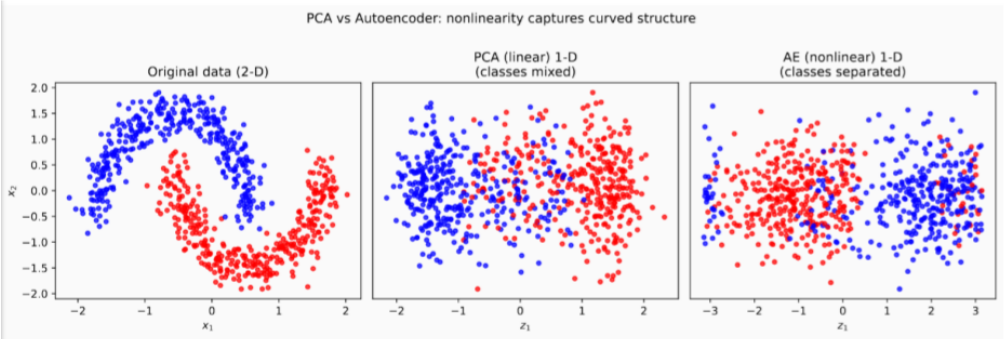
- Use **Adam** (not vanilla SGD)
- **Batch norm** stabilises deep nets
- **Weight tying**: $\boldsymbol{W}_{g_\theta} = \boldsymbol{W}_{f_\phi}^\top$ halves parameters, regularises
- **Early stopping** on val loss
- Output activation: sigmoid for $[0, 1]$, linear for real-valued

# Method Comparison

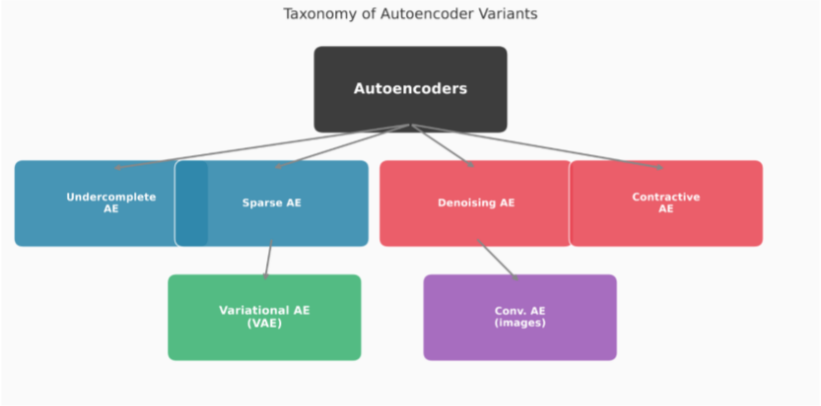
| **Property** | **PCA** | **t-SNE** | **UMAP** | **AE** |
|---|---|---|---|---|
| Parametric (new points) | ✓ | × | part. | ✓ |
| Nonlinear | × | ✓ | ✓ | ✓ |
| Global structure | ✓ | × | part. | depends |
| Generative sampling | × | × | × | VAE: ✓ |
| Scales to $n > 10^5$ | ✓ | slow | ✓ | ✓ |
| Reconstruction map | ✓ | × | × | ✓ |
| Closed-form | ✓ | × | × | × |
| Hyperparam. sensitivity | low | high | medium | medium |

Let us try notebook

# PCA vs Autoencoder: Nonlinearity Matters



# Taxonomy of Autoencoder Variants



# Sparse Autoencoders

**Idea:** penalise non-zero latent activations

$$\mathcal{L}_{\text{sparse}} = \mathcal{L}_{\text{rec}} + \beta \, \|f_\phi(\boldsymbol{x})\|_1$$

**Effect:**

- Most units are zero for any given input
- Only a *sparse subset* activates
- Produces disentangled, interpretable features

**Connection:** sparse dictionary learning, ICA

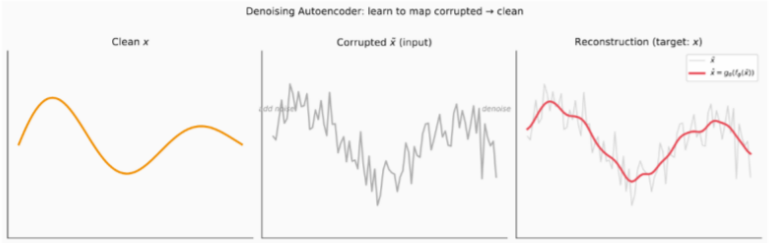
**Use cases**

- Feature interpretability
- Representation for sparse retrieval
- LLM interpretability (Anthropic, OpenAI)

# Denoising Autoencoders (DAE)

**Idea:** corrupt input, reconstruct the *clean* original

$$\tilde{\boldsymbol{x}} = \text{corrupt}(\boldsymbol{x}) \longrightarrow \hat{\boldsymbol{x}} = g_\theta(f_\phi(\tilde{\boldsymbol{x}})), \quad \mathcal{L} = \|\boldsymbol{x} - \hat{\boldsymbol{x}}\|^2$$



# Contractive Autoencoders (CAE)

**Idea:** penalise sensitivity of the encoder to input perturbations

$$\mathcal{L}_{\text{CAE}} = \mathcal{L}_{\text{rec}} + \lambda \left\| \frac{\partial f_\phi(\boldsymbol{x})}{\partial \boldsymbol{x}} \right\|_F^2$$

**Jacobian penalty forces:**

- Encoder is *locally flat*
- Small input changes $\Rightarrow$ small code changes
- Robust to nuisance perturbations

**Geometric intuition**

The encoder *contracts* neighbourhoods in input space onto the learned manifold

**Comparison with DAE:**

Both aim for robustness — DAE via stochastic corruption at training time, CAE via an explicit deterministic Jacobian penalty.

# Variational Autoencoders (VAE)

**Key change:** encoder outputs a *distribution*, not a point

$$q_\phi(\boldsymbol{z}|\boldsymbol{x}) = \mathcal{N}\big(\boldsymbol{\mu}_\phi(\boldsymbol{x}),\, \mathrm{diag}(\boldsymbol{\sigma}_\phi(\boldsymbol{x}))^2\big)$$

**Training objective — ELBO:**

$$\mathcal{L}_{\mathrm{VAE}} = \underbrace{\mathbb{E}_{q_\phi}[\log p_\theta(\boldsymbol{x}|\boldsymbol{z})]}_{\text{reconstructed output}} - \underbrace{\mathrm{KL}\big(q_\phi(\boldsymbol{z}|\boldsymbol{x}) \,\|\, \mathcal{N}(0, I)\big)}_{\text{KL regularisation}}$$

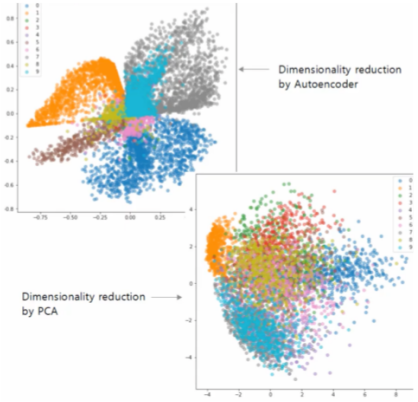
# Variational Autoencoders (VAE)

**Key change:** encoder outputs a *distribution*, not a point

$$q_\phi(\boldsymbol{z}|\boldsymbol{x}) = \mathcal{N}\big(\boldsymbol{\mu}_\phi(\boldsymbol{x}),\, \mathrm{diag}(\boldsymbol{\sigma}_\phi(\boldsymbol{x}))^2\big)$$

**Training objective — ELBO:**

$$\mathcal{L}_{\mathrm{VAE}} = \underbrace{\mathbb{E}_{q_\phi}[\log p_\theta(\boldsymbol{x}|\boldsymbol{z})]}_{\text{reconstructed output}} - \underbrace{\mathrm{KL}\big(q_\phi(\boldsymbol{z}|\boldsymbol{x}) \,\|\, \mathcal{N}(0, I)\big)}_{\text{KL regularisation}}$$


Dimensionality reduction
by Autoencoder
Dimensionality reduction
by PCA

# Reparametrisation Trick (VAE)

## Problem

Backpropagation cannot pass gradients through a *stochastic* node $\boldsymbol{z} \sim q_\phi(\boldsymbol{z}|\boldsymbol{x})$.

## Solution: Reparametrisation

Write $\boldsymbol{z}$ as a *deterministic* function of $\phi$ plus external noise:

$$\boldsymbol{z} = \boldsymbol{\mu}_\phi(\boldsymbol{x}) + \boldsymbol{\sigma}_\phi(\boldsymbol{x}) \odot \boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}(0, I)$$

Now $\nabla_\phi \boldsymbol{z}$ exists — the randomness is "moved outside".

**This enables:**

- Gradient flow through $\boldsymbol{\mu}_\phi$ and $\boldsymbol{\sigma}_\phi$
- Generation by sampling $\boldsymbol{\epsilon} \sim \mathcal{N}(0, I)$
- Smooth interpolation in latent space

## Modern descendants

$\beta$-VAE, VQ-VAE, DALL-E tokeniser, Stable Diffusion latent space

# Applications of Autoencoders

## Anomaly Detection

Train on *normal* data only.
At test time:

$$s(\boldsymbol{x}) = \|\boldsymbol{x} - g_\theta(f_\phi(\boldsymbol{x}))\|^2$$

Anomalies lie off the manifold
$\Rightarrow$ high reconstruction error

## Retrieval

Compact $\boldsymbol{z}$ codes $\Rightarrow$ fast
approx. nearest-neighbour (FAISS)
in low-dimensional latent space

## Visualisation Pipeline

$$\text{raw } (d\text{-D}) \xrightarrow{\text{AE}} z\ (\sim 50\text{-D}) \xrightarrow{\text{UMAP}} 2\text{-D}$$

AE removes noise first;
UMAP then reveals structure cleanly

## Pre-training

Encoder weights $\phi$ initialise
a downstream classifier.
Powerful in low-label regimes.

# Common Pitfalls

## Things that can go wrong

1. **Posterior collapse (VAE)** — KL term dominates; encoder ignored.
   *Fix:* KL annealing (warm-up from $\beta = 0$ to 1)
2. **Over-complete bottleneck** — identity map, no compression.
   *Fix:* choose $k$ by cross-validation
3. **Low MSE $\neq$ good representation** — always evaluate downstream.
4. **Blurry reconstructions** — MSE averages over modes.
   *Fix:* perceptual loss or adversarial (GAN) decoder
5. **No global structure** — AE latent space has no canonical metric.
   *Fix:* pair with UMAP for visualisation
6. **Training instability** — vanishing gradients in deep nets.
   *Fix:* batch normalisation, residual connections

# The Autoencoder Family

| Model | Characterisation |
|---|---|
| Autoencoder (AE) | = PCA + nonlinearity + parametric encoder/decoder |
| Denoising AE (DAE) | = AE + robustness via input corruption |
| Contractive AE (CAE) | = AE + robustness via Jacobian penalty |
| Variational AE (VAE) | = AE + probabilistic latent + generative |
| Convolutional AE | = AE + spatial inductive bias (images) |
| Masked AE (MAE) | = DAE + patch masking + transformer |

## Looking ahead

**MAE** (He et al., 2022): masks 75% of image patches $\Rightarrow$ state-of-the-art vision representations.
**VQ-VAE**: discrete latent codebook $\Rightarrow$ image tokeniser in diffusion & autoregressive models.

# Key Takeaways

1. Autoencoders learn a **parametric, nonlinear** compression: input $\rightarrow$ bottleneck $\rightarrow$ reconstructed output.
2. **PCA is a special case** (linear activations + MSE loss). Nonlinear activations strictly generalise it.
3. The bottleneck dimension $k$ should match the **intrinsic dimensionality** of the data.
4. **DAE** $\rightarrow$ **diffusion**, **VAE** $\rightarrow$ **generation**: small changes to the AE unlock powerful extensions.
5. Always validate representations with a **downstream task** (clustering, classification, retrieval) — not just MSE.
6. Recommended pipeline: **AE** (noise removal, 50-D) $\rightarrow$ **UMAP** (2-D visualisation).

# Further Reading


- **Goodfellow, Bengio & Courville** (2016), *Deep Learning*, Chapter 14: Autoencoders. MIT Press
- **Kingma & Welling** (2014), *Auto-Encoding Variational Bayes*. ICLR 2014
- **Vincent et al.** (2010), *Stacked Denoising Autoencoders*. JMLR 11
- **Rifai et al.** (2011), *Contractive Auto-Encoders*. ICML 2011
- **He et al.** (2022), *Masked Autoencoders Are Scalable Vision Learners*. CVPR 2022
- **van den Oord et al.** (2017), *Neural Discrete Representation Learning* (VQ-VAE). NeurIPS 2017

# Questions?

Autoencoders — Lecture 3

Embeddings & Dimensionality Reduction
Constructor University

Next: practical notebook `autoencoder_embeddings.ipynb`